

QUESTION 1: Interpret the XML Structure

a. Identify the root element.

Answer:

The root element of the XML document is <courses>. It is the outermost element and contains all the individual course records in the document.

b. Identify the repeating record element.

Answer:

The repeating record element is <course>. Each <course> element represents one course offered by the university, and multiple course elements are present under the <courses> element.

c. Identify the attribute used to uniquely identify each course.

Answer:

The attribute used to uniquely identify each course is id. It is associated with the <course> element, for example <course id="C101">. Each course has a different ID such as C101, C102, C103, C104 and C105.

d. Identify the elements that represent numerical information.

Answer:

The elements that represent numerical information are <students> and <credits>. The <students> element represents the number of students enrolled in a course, while the <credits> element represents the number of credits assigned to the course.

e. State whether the XML document is structurally well-formed and justify your answer.

Answer:

Yes, the XML document is structurally well-formed. It contains a single root element <courses>.

all elements have properly matched opening and closing tags, and the elements are correctly nested. The id attributes are also properly written within quotation marks. Therefore, the given XML document follows the basic rules of a well-formed XML document.

QUESTION 2: Write XPath Expressions

```
<?xml version="1.0" encoding="UTF-8"?>
```

```
<courses>
```

```
<course id="C101">
```

```
<code>WEB301</code>
```

```
<name>Web Technology</name>
```

```
<faculty>Dr. Arun</faculty>
```

```
<students>58</students>
```

```
<credits>4</credits>
```

```
<type>Theory</type>
```

```
</course>
```

```
<course id="C102">
```

```
<code>AI302</code>
```

```
<name>Artificial Intelligence</name>
```

```
<faculty>Dr. Meena</faculty>
```

```
<students>72</students>
```

```
<credits>4</credits>
```

```
<type>Theory</type>
```

```
</course>
```

<course id="C103">

<code>WEB303</code>

<name>Web Technology Laboratory</name>

<faculty>Dr. Ravi</faculty>

<students>36</students>

<credits>2</credits>

<type>Practical</type>

</course>

<course id="C104">

<code>ML304</code>

<name>Machine Learning</name>

<faculty>Dr. Priya</faculty>

<students>64</students>

<credits>4</credits>

<type>Theory</type>

</course>

<course id="C105">

<code>DB305</code>

<name>Database Systems</name>

<faculty>Dr. Kumar</faculty>

<students>42</students>

<credits>3</credits>

<type>Theory</type>

</course>

</courses>

a. Select all course records

/courses/course

This XPath selects all the <course> elements present inside the <courses> root element.

b. Select the names of all courses

/courses/course/name

This selects the <name> element from every course record.

c. Select courses having more than 50 students

/courses/course[students > 50]

This selects only those course records where the value of <students> is greater than 50.

d. Select courses carrying 4 credits

/courses/course[credits = 4]

This selects the courses whose <credits> value is exactly 4.

e. Select courses whose type is Theory

/courses/course[type = 'Theory']

This selects all course records where the <type> element contains Theory.

f. Select names of Theory courses having more than 50 students

/courses/course[type = 'Theory' and students > 50]/name

This combines two conditions: the course must be of type Theory and must have more than 50 students.

g. Select faculty members handling courses with at least 4 credits

/courses/course[credits >= 4]/faculty

Here, ≥ 4 means the course must have **4 or more credits**. The expression returns the corresponding faculty names.

h. Select the course whose ID is C104

```
/courses/course[@id = 'C104']
```

The @ symbol is used to access the id attribute of the <course> element.

i. Select the first course

```
/courses/course[1]
```

The [1] selects the first <course> element.

j. Select the last course

```
/courses/course[last()]
```

The last() function selects the final <course> element.

QUESTION 3: XSLT Transformation

```
<?xml version="1.0" encoding="UTF-8"?>
```

```
<xsl:stylesheet version="1.0" xmlns:xsl="http://www.w3.org/1999/XSL/Transform">
```

```
<xsl:template match="/">
```

```
<html>
```

```
<head><title>High Enrollment Courses</title></head>
```

```
<body>
```

```
<h2>High Enrollment Courses</h2>
```

```
<table border="1">
```

```
<tr><th>Course Code</th><th>Course
```

```
Name</th><th>Faculty</th><th>Students</th><th>Credits</th><th>Type</th></tr>
```

```

<xsl:for-each select="courses/course[students > 40]">
  <xsl:sort select="students" data-type="number" order="descending"/>
  <tr>
    <td><xsl:value-of select="code"/></td>
    <td><xsl:value-of select="name"/></td>
    <td><xsl:value-of select="faculty"/></td>
    <td><xsl:value-of select="students"/></td>
    <td><xsl:value-of select="credits"/></td>
    <td><xsl:value-of select="type"/></td>
  </tr>
</xsl:for-each>
</table>
</body>
</html>
</xsl:template>
</xsl:stylesheet>

```

a. Display only courses having more than 40 students.

Answer:

The XSLT uses an xsl:for-each statement with an XPath condition to select only the courses whose student count is greater than 40.

```
<xsl:for-each select="courses/course[students & gt; 40]">
```

This excludes courses having 40 or fewer students.

b. Sort the selected courses by student enrollment in descending order.

Answer:

The xsl:sort element is used to arrange the selected courses according to the number of students in descending order.

```
<xsl:sort select="students"
        data-type="number"
        order="descending"/>
```

Thus, the course with the highest number of students is displayed first.

c. Display the heading "High Enrollment Courses".

Answer:

The heading is displayed using the HTML <h2> element inside the XSLT template.

```
<h2>High Enrollment Courses</h2>
```

d. Display the course details in an HTML table.

Answer:

An HTML table is created using <table>, <tr>, <th> and <td> elements. The xsl:value-of element is used to retrieve the course details from the XML document.

The XSLT transformation converts the selected XML course records into an HTML table and displays them in descending order of student enrollment.

Expected output

Course Code	Course Name	Faculty	Students	Credits	Type
AI302	Artificial Intelligence	Dr. Meena	72	4	Theory
ML304	Machine Learning	Dr. Priya	64	4	Theory

WEB301	Web Technology	Dr. Arun	58	4	Theory
DB305	Database Systems	Dr. Kumar	42	3	Theory

QUESTION 4: Analyze the Course Data – 5 Marks

a. Which course has the highest enrollment?

Answer:

The course with the highest enrollment is Artificial Intelligence (AI302). It has 72 students, which is the highest number of students among all the given courses.

b. Which course has the lowest enrollment?

Answer:

The course with the lowest enrollment is Web Technology Laboratory (WEB303). It has 36 students, which is the lowest enrollment among the given courses.

c. How many Theory courses are present?

Answer:

There are 4 Theory courses in the given XML data. They are:

1. Web Technology
2. Artificial Intelligence
3. Machine Learning
4. Database Systems

Therefore, the total number of Theory courses is **4**.

d. Which courses have exactly 4 credits?

Answer:

The courses having exactly 4 credits are:

1. Web Technology (WEB301)
2. Artificial Intelligence (AI302)
3. Machine Learning (ML304)

Therefore, there are 3 courses carrying 4 credits.

e. Which courses require additional support because they have more than 60 students?

Answer:

The courses having more than 60 students require additional support. They are:

1. Artificial Intelligence (AI302) – 72 students
2. Machine Learning (ML304) – 64 students

Therefore, **2 courses** require additional support.